

# Lab 01 — Commented answers

---

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

---

## Question 1 — "A container is a small VM"

**Answer.** Wrong on the essential point: a container contains **no operating system** and has **no kernel of its own**. It is a process of the host, isolated by *namespaces* and limited by *cgroups*, run by the host kernel.

**Why.** A VM boots a kernel, then an `init`, then dozens of system services (logging, cron, SSH, networking...) before your application even starts: hence the seconds or minutes of boot and the GB of disk. A container boots none of that: the kernel is already running, we just ask it to create namespaces and launch **one** process. The start-up cost is that of a `fork` + `exec`, a few milliseconds; the disk cost is that of the libraries the application actually needs.

### → LINUX

`fork` duplicates the current process, `exec` replaces its content with another program. That is how every Linux process is born, `podman run` included: Podman duplicates itself, the clone enters its namespaces, then executes your command. A container is born exactly like an `ls`.

**Nuance.** The "light VM" intuition is not absurd for a *user*: you do get a `/`, a `hostname`, an IP, a `root`. It becomes dangerous as soon as security is discussed: where a VM's hypervisor is a real boundary, a container shares the kernel — a kernel flaw crosses that boundary. And on Windows the "lightness" is relative: there is a VM, WSL 2, but a single one for all your containers.

### Example.

```
time podman run --rm alpine echo hello      # ~0.3 s, mostly the pull/CLI
podman run -d nginx:alpine                  # ~64 MB image, PID visible on the host via ps
```

## Question 2 — 250 MB "without an OS"

**Answer.** The image contains the **userland** of a distribution: `/bin/sh`, `libc`, `coreutils`, the PostgreSQL binaries, the configuration. What is **absent** is the **kernel** — and with it everything that only exists at boot: bootloader, `initrd`, kernel modules, `systemd`, drivers, hardware management.

**Why.** A Linux program calls the kernel through system calls (`open`, `read`, `fork`). It does not need to ship a kernel, it only needs to find one: the host's will do. So the image only provides what is missing above.

**Nuance.** That is why an "Alpine" image and a "Debian" image can run side by side on the same Ubuntu WSL: three userlands, one kernel. And it is also why a Linux image does not run on a Windows kernel — hence WSL.

### Example.

```
podman run --rm alpine cat /etc/os-release | head -1  # NAME="Alpine Linux"
uname -r                                              # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r                      # STRICTLY the same kernel
```

### Question 3 — Two nginx containers, two different `ps`

**Answer.** The `pid namespace`. Each container gets its own PID table: its main process is number 1 there and it cannot see any outside PID.

**Why.** Seeing a process is a prerequisite for acting on it ( `kill` , `/proc/<pid>` ). By removing visibility, the kernel effectively removes the ability to do harm through that route. On the host these processes do exist, with real PIDs.

**Nuance.** It is not security "in the strict sense" because it is **not an impassable boundary**: it is a visibility restriction enforced by the same kernel the container uses. A container started with `--pid=host` or `--privileged` gets the full view back, and a kernel flaw bypasses the mechanism. The namespace isolates; it does not defend. Rootless, the `user` namespace adds a real barrier of *rights* on top of that barrier of *view*.

**Example.**

```
podman run -d --name web nginx:alpine
podman exec web ps -o pid,comm          # PID 1 = nginx, then its workers
ps -ef | grep -c "[n]ginx"             # on the host: the processes are there
podman run --rm --pid=host alpine ps | head # isolation removed: the whole WSL
podman rm -f -t 0 web
```

### Question 4 — Cannot connect to the Docker daemon

**Answer.** On your colleague's machine, the client worked: it printed its version, then tried to open the `/var/run/docker.sock` socket to query the **server**, and failed. Two likely causes: (1) the daemon is not started, (2) their user has no permission on the socket. For you, that message is impossible: Podman has **no daemon** and contacts no socket; each command does the work itself, under your user.

**Why.** Every Docker command is a network call to `dockerd` . Changing the command will not change anything, since nobody has read it yet: the problem is upstream. Podman is an ordinary program: if it starts, it works.

**Nuance.** There are two cases where Podman *does* have a server: `podman --remote` (or the `CONTAINER_HOST` variable) talking to a remote `podman system service` , and `podman machine` on Windows/macOS, where the Windows client talks to a VM. You would then see `unable to connect to Podman socket` . Under WSL with Podman installed inside Ubuntu, you are in neither case.

**Example.**

```
# Docker side, the diagnosis:
systemctl status docker          # cause 1: inactive (dead)
ls -l /var/run/docker.sock       # srw-rw---- root docker: you must be in the docker group
# Podman side, proof that there is nothing to contact:
podman version                   # a single "Client" block
podman --remote version          # Error: unable to connect to Podman socket ...
```

### Question 5 — "An image does not run"

**Answer.** An image is a set of read-only files plus metadata. There is no process, no state, nothing to schedule: it is as inert as a `.zip` with an instruction sheet.

**Why.** At `podman run`, the engine adds three things: (1) a thin **writable layer** on top of the read-only layers, (2) a set of **namespaces and cgroups**, (3) the **execution** of the command written in the image metadata ( `ENTRYPOINT / CMD` ), through the `crun` runtime. The result of that assembly is the container.

**Nuance.** The *created* container and the *started* container are two distinct steps: `podman create` does everything except launching the process, `podman start` launches it. `podman run` is simply `create` + `start` (+ `pull` if the image is missing).

### Example.

```
podman create --name tmp alpine sleep 30    # container created, no process
podman ps -a --filter name=tmp              # STATUS: Created
podman start tmp && podman ps                # STATUS: Up -> now it runs
podman rm -f -t 0 tmp
```

## Question 6 — PostgreSQL data after a `podman rm`

**Answer.** No, the data is gone. And no, the image was **not** modified: it is strictly identical before and after.

**Why.** A container's writes (through *copy-on-write*) land in its private writable layer. `podman rm` removes the container **and** that layer. The image layers are read-only: nothing a container does can alter them — that is what guarantees two containers of the same image start from the same state.

**Nuance.** Two important corrections. First, `podman stop` destroys nothing: a stopped container keeps its layer, and `podman start` finds the data again. It is `rm` that destroys. Second, the official `postgres` image declares `/var/lib/postgresql/data` as a `VOLUME`: the engine then creates an **anonymous** volume that survives the `rm` — but without a name it is almost impossible to find again. In practice the data is considered lost. The named volume is the subject of lab 06.

### Example.

```
podman run -d --name c1 alpine sleep 600
podman exec c1 sh -c 'echo x > /mark.txt'
podman rm -f -t 0 c1
podman run --rm alpine ls /mark.txt    # No such file or directory: the image is intact
```

## Question 7 — Ten containers, how much disk?

**Answer.** A few dozen kilobytes in total — not  $10 \times 210$  MB. Each container only costs its writable layer, empty at first, plus a few configuration files ( `hostname`, `resolv.conf` ...).

**Why.** The image layers are **shared** read-only by all containers created from them. The `overlay` storage driver stacks those layers and one empty writable layer per container; a file is only copied into that layer when it is modified (*copy-on-write*).

**Nuance.** The answer changes if each container writes a lot (logs, temporary files): every modification of an image file copies it entirely into the container's layer. And `podman ps -s` shows both figures: the own size and the "virtual" size.

### Example.

```
for i in 1 2 3; do podman run -d --name t$i alpine sleep 600; done
podman ps -s --format 'table {{.Names}}\t{{.Size}}' # 11.4kB (virtual 8.72MB) each
podman rm -f -t 0 t1 t2 t3
```

## Question 8 — `root` inside, `1000` on the host

**Answer.** The `user` namespace maps the container's identifiers onto the host's: the container's UID 0 is your UID 1000. That "root" has, towards the kernel and the host's files, only your rights: an attempt to write to `/etc/shadow` mounted from the host fails with `Permission denied`, exactly as if you did it yourself.

**Why.** The kernel checks permissions with the **real** identity (host side), not with the identity displayed in the namespace. The container's UIDs 1 to 65536 are mapped onto a reserved range in `/etc/subuid` (`100000-165535`) that has no rights on your files.

**Nuance.** That root *is* root **inside** its namespaces: it can install packages, change the permissions of the image's files, listen on the container's port 80. What it cannot do is cross the boundary. Practical corollary: a file created by the container under UID 999 (the `postgres` user) appears on your host with UID 100998 — the classic *bind mount* trap in rootless mode (lab 06).

### Example.

```
podman top watcher user,huser # root / 1000
podman unshare cat /proc/self/uid_map # 0 -> 1000 (1), 1 -> 100000 (65536)
podman run --rm -v /etc:/host alpine sh -c 'echo x >> /host/shadow' # Permission denied
```

## Question 9 — The `docker` group and `sudo`

**Answer.** Being a member of the `docker` group grants the right to write to `/var/run/docker.sock`, hence to have anything executed by a daemon running as **root**: `docker run -v /:/host --privileged` gives the whole host. It is `sudo` without a password, without logging and without limit. With rootless Podman there is neither a root daemon nor a socket: the user cannot do anything more than they already could, and the rule has no object any more.

**Why.** Auditing requires knowing *who* did *what*. A command sent through the socket is executed by `dockerd`, under the `root` identity, with no trace tied to the user. `sudo docker ...` at least leaves a line in `auth.log`. Rootless Podman goes further: the container is a process of the user, visible and attributable in `ps`.

**Nuance.** Rootless Podman has a cost: no port < 1024 without tuning, a slightly slower user-space network, some mounts and options forbidden. In production you also meet *rootful* Podman (`sudo podman`), which then brings back the same precautions as Docker.

### Example.

```
# What the docker group allows (DO NOT do this on a shared machine):
docker run --rm -v /:/host alpine cat /host/etc/shadow # readable: the daemon is root
# The same thing under rootless Podman:
podman run --rm -v /:/host alpine cat /host/etc/shadow # Permission denied
```

## Question 10 — Long form, short form

**Answer.**

| Shortcut                             | Long form                                 | Object    |
|--------------------------------------|-------------------------------------------|-----------|
| <code>podman ps -a</code>            | <code>podman container ls -a</code>       | container |
| <code>podman images</code>           | <code>podman image ls</code>              | image     |
| <code>podman rmi nginx:alpine</code> | <code>podman image rm nginx:alpine</code> | image     |
| <code>podman rm web</code>           | <code>podman container rm web</code>      | container |

**Why.** `ps` and `images` date from the first Docker versions (2013), when the CLI had no objects yet: `ps` imitated the Unix command of the same name, `images` was a plural. The `object action` grammar arrived in 2017 (Docker 1.13), and Podman took it over as is. The shortcuts are kept so nothing breaks.

**Nuance.** The long form is the only complete one: `podman container ls`, `podman image ls`, `podman volume ls`, `podman network ls`, `podman pod ls` follow the same pattern, whereas `podman ps` has no equivalent for volumes. In scripts, prefer the long form.

**Example.**

```
podman container ls -a --format '{{.Names}}'
podman image ls --format '{{.Repository}}:{{.Tag}}'
```

## Question 11 — Two `uname -r`, two hosts

**Answer.** `uname -r` is a system call: the value comes from the **kernel**, never from the image. Under WSL, the kernel is `microsoft-standard-WSL2`, compiled by Microsoft for the WSL VM; on a native Ubuntu server, it is the `generic` kernel from the Ubuntu package. A container shows the kernel of the machine that runs it, whatever the image.

**Why.** The container is a process of the host kernel; there is no kernel in the image (question 2). On Windows that host is not Windows but the WSL 2 VM.

**Nuance.** "Lightweight" remains true: the WSL VM is **unique**, started once, and shared by all your containers; those remain processes that start in milliseconds. What is no longer true is "no VM at all". Practical consequences: the available RAM is WSL's (`.wslconfig`), and Windows files (`/mnt/c/...`) mounted into a container are slow, because they cross the VM ↔ Windows boundary. Work inside the Linux file system (`~`).

**Example.**

```
uname -r                # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r    # identical
podman info --format '{{.Host.Kernel}} {{.Host.MemTotal}}' # the RAM as seen by WSL
```

## Question 12 — Infinite loop and RAM

**Answer.** No: the `pid` namespace only hides processes. The mechanism that protects the neighbours is the memory **cgroup**, enabled by `--memory`. Without a limit, the container consumes

all available RAM; when the kernel has nothing left, the **OOM killer** kills a process of its choosing — not necessarily the culprit.

**Why.** Namespaces and cgroups are two independent mechanisms: one isolates the *view*, the other caps *consumption*. With `--memory=512m`, exceeding the limit kills only the container's process (`Exited (137)`, `OOMKilled: true`), and the rest of the machine does not notice.

**Nuance.** Rootless, `--memory` is only possible if `systemd` delegates the `memory` controller to your user — which is the case on Ubuntu WSL once `systemd=true` is enabled. And for Java, a cgroup limit is only useful if the JVM respects it: since Java 10 it reads the cgroup automatically (`-XX:MaxRAMPercentage`), but an `-Xmx` set too high by hand will exceed it anyway.

### Example.

```
podman run -d --name limited --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limited      # MEM USAGE / LIMIT: ... / 134.2MB
podman inspect --format '{{.State.OOMKilled}}' limited  # false – for now
podman rm -f -t 0 limited
```

## Question 13 — Promoting an image without rebuilding it

**Answer.** Two properties: **immutability** (an image identified by its digest never changes) and the **OCI standard** (Docker and Podman produce and read exactly the same format). What was tested in staging is, bit for bit, what goes to production, whatever the engine. Rebuilding at each stage breaks that guarantee: two builds of the same code do not necessarily give the same image.

**Why.** A build depends on the moment: `apt-get install` takes the version of the day, `FROM eclipse-temurin:21-jre` follows a moving tag, Maven resolves version ranges. Between the staging build and the production build, a dependency may have changed — and the staging validation is worth nothing any more.

**Nuance.** Promotion is not done by re-tagging `latest` but by referencing the **digest** (`api@sha256:...`) or an immutable tag (`api:1.4.2`). Lab 02 comes back to it. And the engine matters little: a `podman pull` of an image pushed by `docker push` is a banal case.

### Example.

```
podman image inspect --format '{{.Digest}}' registry.internal/api:1.4.2
# Same digest on the workstation (Podman), in staging (Docker) and in production (rootful Podman).
```

## Question 14 — `alias docker=podman`

**Answer.** True without reservation for: (1) the whole image cycle — `build`, `pull`, `push`, `tag`, `images`, `history`, `inspect`, Dockerfiles; (2) the container life cycle — `run`, `ps`, `logs`, `exec`, `stop`, `rm` with their options. False or different: (a) **no daemon** — no `docker.sock`, `--restart=always` does not survive a reboot without `systemd`, `podman rm -f` waits 10 s; (b) **rootless** — no port < 1024 without tuning, IP addresses absent with `pasta`, shifted UIDs on `bind mount` files, `--memory` conditional on cgroup delegation.

**Why.** Podman copied Docker's *surface* (the CLI, the format) but not its *architecture*. Everything that depends only on the surface is identical; everything that touches "who executes, with which rights, supervised by whom" diverges.

**Nuance.** The differences are not defects: each is the counterpart of a security choice. And Docker Compose works with Podman ( `podman compose` , lab 09), at the price of a few configuration lines.

**Example.**

```
alias docker=podman
docker run -d --name web -p 8080:80 nginx:alpine    # identical
docker run -d --name w80 -p 80:80 nginx:alpine      # Error: pasta failed ... Listen failed for
HOST TCP port */80: Permission denied
podman rm -f -t 0 web
```